Themes & Publication-Quality Styling

Workshop 2 — Module 6

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. What a theme controls: the non-data layer
2. Five built-in ggplot2 themes
3. The four `element_*()` building blocks
4. Writing a reusable custom theme function
5. Publication themes via `ggthemes`
6. Python equivalents: `rcParams`, `plt.style.use()`
7. Export formats and resolution for publication

Learning Outcome
You will transform any chart from default styling to publication quality using `theme()` elements and create a reusable branded theme function.

What a Theme Controls

theme() Controls Every Non-Data Element

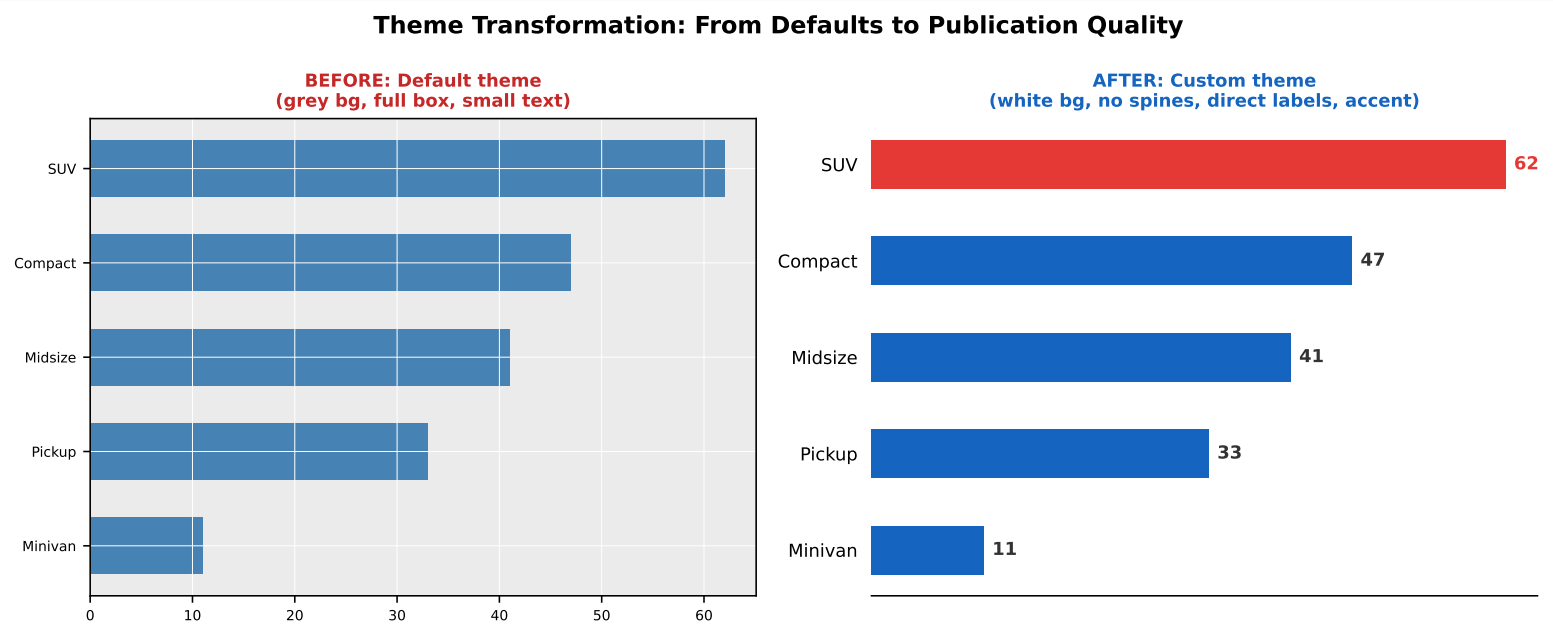
theme() Controls Every Non-Data Element

`plot.title / plot.subtitle`	Title text: face, size, color, margin
`axis.title.x / axis.title.y`	Axis label text: face, size, color
`axis.text.x / axis.text.y`	Tick label text: size, angle, hjust
`panel.background`	Plot area fill colour
`panel.grid.major / minor`	Gridlines: colour, linewidth, linetype
`strip.text / strip.background`	Facet strip styling
`legend.position / legend.key`	Legend placement and key size
`plot.margin`	Whitespace around entire plot

Pro-Tip

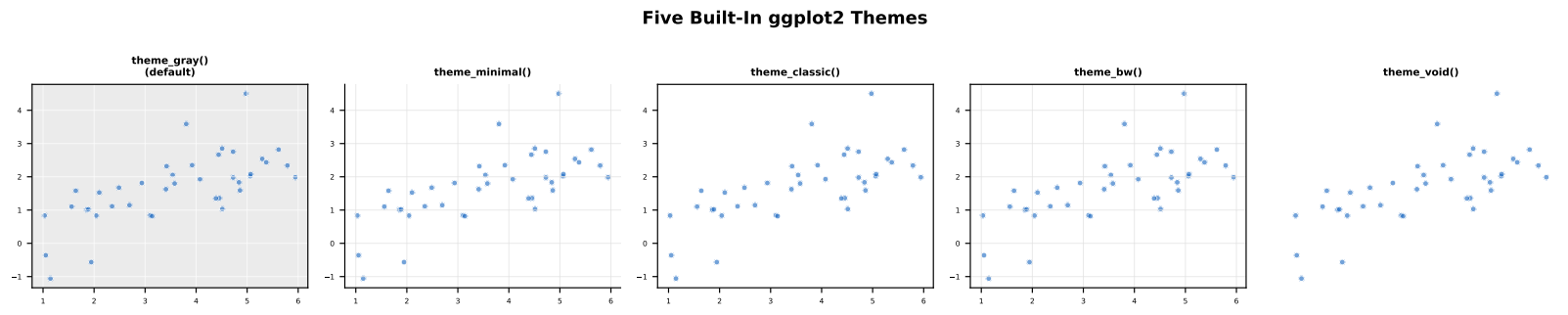
A theme modifies *appearance*, never *data*. You can change the theme without changing a single data point, aesthetic mapping, or statistic. This is the grammar's separation of content from presentation.

Before & After: The Power of theme()



Built-In Themes

Five Core ggplot2 Themes



Data Insight

`theme_minimal()` is the recommended starting point: white background, subtle gridlines, no box frame. Add `base_size` for global font scaling: `theme_minimal(base_size = 14)` for presentations; `base_size = 9` for journal figures.

Choosing a Base Theme

Theme	Best For
`theme_minimal()`	Default recommendation: clean, modern
`theme_classic()`	Journal articles with axis-line style
`theme_bw()`	Print-safe with panel border
`theme_void()`	Maps, network diagrams, pie charts
`theme_gray()`	ggplot2 default (avoid for publication)

Start with one of these, then customize individual elements with `theme(...)`. Never build a theme from scratch.

The element_*() System

Four Building Blocks

Four element_*() Functions: The Building Blocks of theme()

Function	Key Arguments	Used For
`element_text()`	face, size, colour, angle, hjust, vjust, margin	Titles, labels, strip text
`element_line()`	colour, linewidth, linetype, lineend	Gridlines, axis lines, ticks
`element_rect()`	fill, colour, linewidth, linetype	Backgrounds, legend key, strip bg
`element_blank()`	(no arguments)	Remove element entirely

Common theme() Customisations

```r
theme_minimal(base_size = 11) +
theme(
  # Title: bold, 14pt, left-aligned
  plot.title = element_text(face = "bold", size = 14, hjust = 0),
  plot.subtitle = element_text(color = "grey50", size = 10),
  plot.caption = element_text(face = "italic", size = 8, hjust = 0),

  # Axes: remove top/right gridlines
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),

  # Axis labels: slight margin
  axis.title = element_text(face = "bold", size = 10),
  axis.text = element_text(size = 8),

  # Legend: bottom, horizontal
  legend.position = "bottom",
  legend.title = element_text(face = "bold", size = 9),

  # Margins: breathing room
  plot.margin = margin(t = 10, r = 15, b = 10, l = 10)
)
```

Removing Elements with element_blank()

```
# Remove specific elements entirely
theme(
  # Remove y-axis gridlines (for horizontal bar charts)
  panel.grid.major.y = element_blank(),

  # Remove axis ticks and text on one side
  axis.ticks.x = element_blank(),
  axis.text.x = element_blank(),

  # Remove legend
  legend.position = "none",

  # Remove panel border
  panel.border = element_blank(),

  # Remove strip background (facet headers)
  strip.background = element_blank()
)
```

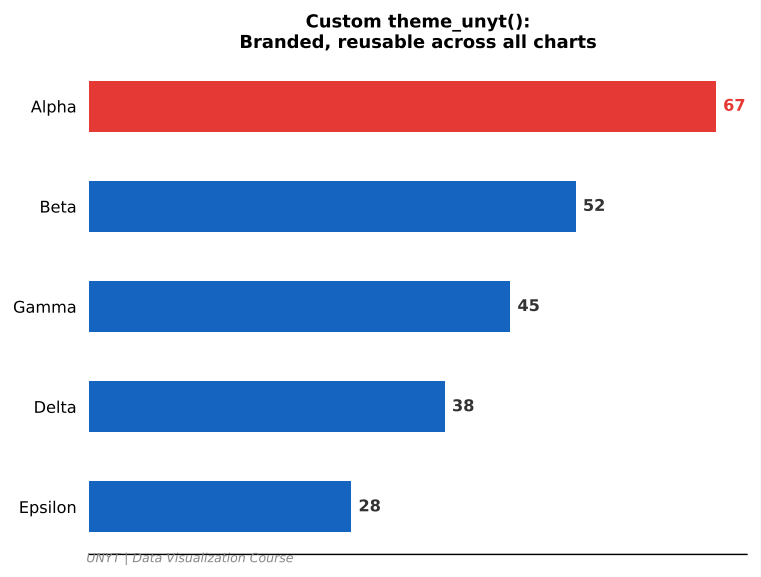
Pro-Tip

`element_blank()` is the Tufte eraser: every non-data element you remove increases the data-ink ratio (W01-M02).

Custom Reusable Themes

Writing theme_unyt(): Your Branded Theme

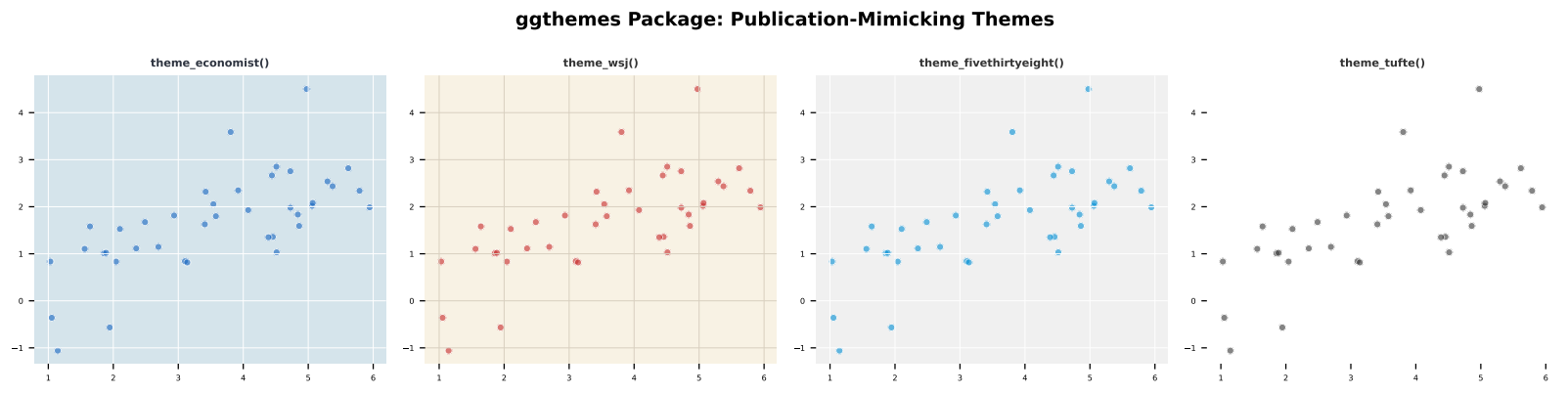
```r
# Define once, use everywhere
theme_unyt <- function(
    base_size = 11) {
  theme_minimal(
    base_size = base_size) +
  theme(
    plot.title = element_text(
      face = "bold", size = rel
          (1.3)),
    plot.subtitle = element_text(
      color = "grey50"),
    plot.caption = element_text(
      face = "italic", hjust = 0),
    panel.grid.major.x =
      element_blank(),
    panel.grid.minor =
      element_blank(),
    legend.position = "bottom",
    plot.margin = margin
        (10,15,10,10)
  )
}

# Usage: one line replaces 15+
ggplot(df, aes(x, y)) +
```



A custom theme function ensures brand consistency across all charts in a project. Define it once in a utility script, source it in every analysis.

ggthemes: Publication Styles

Mimicking Publication Styles



Data Insight

The `ggthemes` package provides themes that mimic *The Economist*, *WSJ*, *FiveThirtyEight*, and Tufte's style. Install with `install.packages("ggthemes")`. These are useful for matching a target publication's visual identity.

Python Equivalents

rcParams: Python's Global Theme System

Python: plt.rcParams — Global Style Configuration

rcParams Key	Value	Effect
`figure.figsize`	`(7, 5)`	Default figure dimensions
`figure.dpi`	`150`	Screen resolution
`font.family`	`'sans-serif'`	Global font family
`font.size`	`11`	Base font size
`axes.spines.top`	`False`	Remove top spine
`axes.spines.right`	`False`	Remove right spine
`axes.facecolor`	`'white'`	Panel background
`axes.grid`	`True`	Show gridlines
`grid.alpha`	`0.15`	Grid transparency

Three Python Approaches to Theming

1. rcParams (global)

```
plt.rcParams.update({
    "figure.figsize": (7, 5),
    "figure.dpi": 150,
    "font.size": 11,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.15,
})
```

2. plt.style.use()

```
# Built-in styles
plt.style.use("seaborn-v0_8-
    whitegrid")
plt.style.use("ggplot")
plt.style.use("bmh")
```

3. Per-chart manual

```
fig, ax = plt.subplots()
ax.scatter(x, y)

# Manual Tufte cleanup
ax.spines["top"].set_visible(
    False)
ax.spines["right"].set_visible(
    False)
ax.set_facecolor("white")
ax.grid(alpha=0.12)
ax.tick_params(labelsize=8)
ax.set_title("Title",
    fontweight="bold", fontsize
        =12)
fig.text(0.12, 0.01,
    "Source: ...",
    fontsize=6, color="#AAA")
```

plotnine uses `+ theme_minimal()` identical to ggplot2.

Export for Publication

Export Format Guide

Export Format Guide: Vector > Raster > Never JPEG

Format	Type	Use Case	R	Python
PDF	Vector	Scalable, print-ready	`ggsave('p.pdf', w=7, h=5)`	`fig.savefig('p.pdf')`
SVG	Vector	Web, editable in Illustrator	`ggsave('p.svg', w=7, h=5)`	`fig.savefig('p.svg')`
PNG	Raster	Web, presentations, >=300 dpi	`ggsave('p.png', dpi=300)`	`fig.savefig('p.png', dpi=300)`
JPEG	Raster	Photos only — NEVER for charts	`(avoid)`	`(avoid)`

Export in R vs Python

R (ggsave)

```r
# Vector (always preferred)
ggsave("chart.pdf",
  plot = p,
  width = 7, height = 5,
  device = cairo_pdf)

# High-res raster
ggsave("chart.png",
  plot = p,
  width = 7, height = 5,
  dpi = 300)

# SVG for web
ggsave("chart.svg",
  plot = p,
  width = 7, height = 5)
```

Python (savefig)

```python
# Vector (always preferred)
fig.savefig("chart.pdf",
    bbox_inches="tight")

# High-res raster
fig.savefig("chart.png",
    dpi=300,
    bbox_inches="tight")

# SVG for web
fig.savefig("chart.svg",
    bbox_inches="tight")

# ALWAYS close after saving
plt.close()
```

Pro-Tip

Always use `bbox_inches="tight"` in Python to prevent label clipping. In R, `cairo_pdf` produces better anti-aliased text than the default PDF device.

Synthesis

Theme Workflow

1. Start with a **base theme**: `theme_minimal(base_size)`
2. Remove non-data ink with **element_blank()**: minor gridlines, unnecessary axis ticks, panel borders
3. Customise text with **element_text()**: bold titles, italic captions, sized axis labels
4. Wrap into a **custom function**: `theme_unyt()` for brand consistency
5. Export as **PDF** (vector) for print, **PNG 300dpi** for screen, **SVG** for web
6. Never export as **JPEG** (lossy compression destroys sharp lines and text)

Key Takeaways

1. **theme()** controls all non-data elements: titles, axes, gridlines, backgrounds, legends, margins.
2. Four building blocks: **element_text()**, **element_line()**, **element_rect()**, **element_blank()**.
3. Start with **theme_minimal()** and customise; never build from scratch.
4. Create a **reusable theme function** for project-wide brand consistency.
5. In Python: **rcParams** for global settings, manual spine removal per chart, or `plt.style.use()`.
6. Export: **PDF/SVG** (vector) ¿ **PNG 300dpi** (raster) ¿ never JPEG.

Essential Readings

- Wickham, H. (2016). *ggplot2*, Chapter 8 (Themes).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 22 (Titles, Captions, and Tables).
- Chang, W. (2024). *R Graphics Cookbook*, 2nd ed., Chapter 9 (Controlling the Overall Appearance).
- ggthemes documentation: `https://jrnold.github.io/ggthemes/`

Next Module: seaborn & plotnine for Python (W02-M07)